

You are Codex operating as an autonomous senior full-stack engineer, geospatial engineer, product designer, security reviewer, and test engineer.

Build the complete application described below in the current directory. Assume the directory is empty unless inspection shows otherwise.

Do not stop after planning or scaffolding. Do not return pseudocode, partial implementations, TODO-only placeholders, or mock screens. Inspect the environment, create the project, implement the application, run all verification commands, diagnose failures, and fix them. Continue until the project builds and the test suite passes.

Do not ask the user follow-up questions. Make reasonable engineering decisions consistent with this specification. Document any unavoidable limitation in the README rather than blocking implementation.

1. Product

Build a polished, mobile-first progressive web application for runners who are visiting, traveling in, or have recently moved to an unfamiliar place and want a circular running route beginning near their current location.

Use the working product name **LoopRoute**. Centralize this name in one configuration file so it can easily be changed later.

The primary workflow is:

1. The app opens on a phone.
2. It immediately requests the user's current GPS position.
3. It centers a map on that position.
4. The user selects a target running distance.
5. The app requests three approximately circular pedestrian routes beginning and ending near that position.
6. It ranks the candidates based on distance accuracy, repeated segments, closure, and overall loop shape.
7. The user selects a route.
8. The user can view the route, export it as GPX, share a reproducible route link, or enter a basic live-follow mode that displays their current position on the route.

The application must be globally usable wherever the routing provider and OpenStreetMap data have coverage.

This is a free hobby project. It must require no paid infrastructure and must not silently introduce services that require billing information.

2. Non-negotiable cost constraints

The project must operate using free tiers and free/open data:

- Cloudflare Pages for static hosting.
- Cloudflare Pages Functions for thin API proxy endpoints.
- MapLibre GL JS for map rendering.
- OpenFreeMap for the public vector basemap.
- OpenStreetMap-derived data and appropriate attribution.
- The free openrouteservice API for route generation and geocoding.
- Browser-side computation for route analysis and ranking.

Do not:

- Add a paid database.
- Add a paid map provider.
- Add a paid authentication service.
- Add a server or container.
- Attempt to self-host a global routing graph.
- Perform expensive geospatial route scoring inside Cloudflare Functions.
- expose the openrouteservice API key in frontend code, generated JavaScript, HTML, source maps, query strings visible to the browser, or committed files.

Cloudflare Functions must remain thin enough for the Workers free-plan CPU limits. They may validate a request, construct an upstream request, make a network fetch, apply safe headers, and return or stream the upstream response. All substantial geometry analysis must happen in the browser.

The practical upstream quota is limited. One initial generation should normally consume exactly three routing calls. Automatic retries must be tightly capped.

3. Technology stack

Use:

- React
- TypeScript with strict mode
- Vite
- MapLibre GL JS
- Tailwind CSS
- vite-plugin-pwa
- Modular Turf packages only where useful; do not import the entire `@turf/turf` bundle
- Vitest
- React Testing Library
- Playwright
- ESLint
- Prettier
- Cloudflare Pages Functions
- Wrangler
- npm

Use current stable package versions that are compatible with one another. Do not pin obsolete versions merely because they are familiar.

Avoid an unnecessarily large component framework. Build accessible application components directly with React and Tailwind. Lucide React may be used for icons.

4. Repository and deliverables

Create a complete repository containing at least:

- Application source code
- Cloudflare Pages Functions
- A PWA manifest and icons
- Service-worker configuration
- Unit tests
- Component tests
- End-to-end tests
- ESLint and Prettier configuration
- TypeScript configuration
- Wrangler/Cloudflare configuration as appropriate
- `.dev.vars.example`
- `.env.example` if useful
- `.gitignore`
- `_headers` or equivalent security-header configuration
- GitHub Actions CI
- A comprehensive README
- An `AGENTS.md` summarizing project conventions for future Codex work
- License file using MIT unless an included dependency or asset requires otherwise

Initialize Git if the directory is not already a repository. Never commit or fabricate an actual API key.

5. Application architecture

Use this high-level architecture:

Browser

The browser is responsible for:

- GPS permission and positioning
- Map rendering
- Start-marker interaction
- Distance and preference controls
- Making requests to same-origin API endpoints
- Generating three seeds and requesting three route candidates
- Parsing routing responses
- Route normalization
- Distance validation
- Repeat detection

- Candidate deduplication
- Candidate scoring and ranking
- Elevation and surface summaries
- GPX generation
- Share-link generation
- Local route persistence
- Live GPS follow mode
- All route-related UI state

Cloudflare Pages Functions

Pages Functions are responsible only for:

- Keeping `ORS_API_KEY` secret
- Strictly validating user-controlled request data
- Rejecting oversized or malformed requests
- Calling the allow-listed openrouteservice endpoints
- Returning safe, normalized HTTP errors
- Applying `Cache-Control: no-store`
- Avoiding logging coordinates, addresses, request bodies, or API keys

Do not store precise locations or generated routes on the server.

External providers

Create provider abstractions so another provider could be substituted later:

- `RoutingProvider`
- `GeocodingProvider`
- `MapStyleProvider` or centralized map configuration

Implement:

- `OpenRouteServiceRoutingProvider`
- `OpenRouteServiceGeocodingProvider`
- OpenFreeMap style configuration

Keep provider-specific response parsing isolated from React components.

6. Cloudflare API endpoints

Implement these same-origin endpoints:

`POST /api/route`

Expected browser request:

```
interface RouteRequest {
  start: {
```

```

    latitude: number;
    longitude: number;
  };
  targetDistanceMeters: number;
  seed: number;
  mode: "road" | "mixed" | "trail";
  avoidSteps: boolean;
}

```

Validate all fields manually or with a very lightweight shared validator:

- Latitude: -90 through 90
- Longitude: -180 through 180
- Target distance: 1,000 through 100,000 metres
- Seed: positive safe integer within a documented upper bound
- Mode: exact enum value
- Request content type: JSON
- Request body size: reject clearly excessive bodies
- Reject unknown or structurally invalid input with HTTP 400

Map the modes to openrouteservice profiles and preferences:

Road

- Profile: `foot-walking`
- Avoid ferries and fords
- Avoid steps when requested
- Little or no green weighting
- Apply quiet weighting conservatively
- This is a preference, not a guarantee of paved surfaces

Mixed

- Profile: `foot-walking`
- Avoid ferries and fords
- Avoid steps when requested
- Prefer green and quiet ways using supported public-API weighting options

Trail

- Profile: `foot-hiking`
- Avoid ferries and fords
- Avoid steps only when requested
- Prefer green ways
- Clearly label that trail quality depends on map data

Call the openrouteservice GeoJSON directions endpoint for the selected pedestrian profile.

The upstream request must use one starting coordinate and round-trip options:

- Target length in metres
- A route-shaping point count based on distance

- The supplied random seed

Use a bounded point-count function such as:

- Under 8 km: 5 points
- 8 km through under 15 km: 7 points
- 15 km through under 30 km: 9 points
- 30 km and longer: 12 points

Keep this function centralized and tested.

Request, where supported:

- GeoJSON geometry
- Elevation
- Route instructions
- Route summary
- `surface`
- `waytype`
- `steepness`
- `osmid`
- `green`
- `noise`

Request only fields useful to the app.

Use the API key in the server-side authorization mechanism supported by openrouteservice.

Never send the key back to the client.

Return the upstream GeoJSON response without conducting route scoring in the Function. Avoid unnecessary JSON serialization cycles if a safe streamed response is possible. Preserve useful upstream status codes, but map errors into safe application errors when necessary.

`GET /api/geocode`

Accept:

- `q`: submitted search text
- Optional `lat` and `lng` focus coordinates

Requirements:

- Search occurs only when the user submits the form.
- Do not implement API-backed autocomplete on each keystroke.
- Minimum trimmed query length: 3 characters.
- Maximum query length: 200 characters.
- Return at most five results.
- Validate optional coordinates.
- Use openrouteservice's geocoding service.

- Strip unneeded provider fields before returning results if this can be done within a lightweight Function; otherwise normalize them in the browser.
- Set `Cache-Control: no-store`.
- Do not log search strings.

Each result exposed to the UI should ultimately normalize to:

```
interface GeocodingResult {
  id: string;
  label: string;
  locality?: string;
  region?: string;
  country?: string;
  latitude: number;
  longitude: number;
}
```

`GET /api/health`

Return a small JSON response indicating:

- The app backend is reachable.
- Whether `ORS_API_KEY` is configured, without revealing it.
- No external API request is required.

7. Location behavior

On a normal application launch:

1. Render the app shell immediately.
2. Request geolocation immediately after the first render.
3. Show a clear locating state.
4. Use high accuracy where sensible, with a finite timeout and maximum-age setting.
5. Once resolved, center the map and place a draggable start marker.
6. Display the reported GPS accuracy.

Handle:

- Permission denied
- Position unavailable
- Timeout
- Unsupported browser
- Insecure context
- Low-accuracy position

If location fails, keep the app usable through:

- Address/place search
- Dragging the start marker

- Tapping or long-pressing the map to set the start point
- Restoring the last local start point when available

Do not continuously track the user until live-follow mode is explicitly entered.

When a route is returned, treat its first geometry coordinate as the provider-snapped start point. If it differs from the requested point by more than 10 metres:

- Display the snapped start marker.
- Draw a subtle dashed connector between the requested point and snapped start.
- Explain that the route was snapped to the nearest routable path.

8. Distance controls

Offer these presets:

- 3 km
- 5 km
- 7.5 km
- 10 km
- 15 km
- Half marathon: 21.0975 km
- 30 km
- Marathon: 42.195 km
- Custom

The custom input must allow 1–100 km.

Keep all internal measurements in metres.

Provide a kilometres/miles display setting:

- Default according to browser locale when reasonably inferable
- Otherwise default to kilometres
- Changing display units must not alter the underlying target distance unexpectedly
- Race presets retain recognizable names

Target tolerance:

- Preferred: within 2%
- Acceptable: within 5%
- A candidate outside 5% may be shown only when not enough acceptable routes are available, and it must be clearly marked

Always show the actual calculated distance.

9. Candidate generation and quota conservation

For the initial request:

1. Generate three distinct positive integer seeds using `crypto.getRandomValues`.
2. Make three independent calls to `/api/route`.
3. Limit concurrency to three.
4. Parse and score successful responses.
5. Deduplicate geometrically similar routes.
6. Rank the remaining routes.
7. Show up to three route cards.

Automatic retries:

- Retry only when a request fails, falls outside 5%, or produces a near-duplicate.
- Permit no more than two additional route calls across the entire initial generation action.
- Therefore, an initial action may consume at most five routing calls.
- Do not retry HTTP 401 or configuration errors.
- Respect HTTP 429 and surface a quota/busy message.
- Use exponential delay only for transient server failures, not for deterministic route failures.
- Abort stale requests when the user changes the start or distance.
- Use `AbortController`.

The **Generate another** action:

- Makes exactly one new routing call with a new seed.
- Does not regenerate all three routes.
- Replaces the lowest-ranked unselected candidate.
- If all three candidates are selected or otherwise protected, replace the currently selected candidate only after a lightweight confirmation.
- Avoid adding an unlimited list of route cards.

Disable generation while the start point is missing.

10. Route normalization

Create a normalized model independent of the provider:

```
interface NormalizedRoute {
  id: string;
  seed: number;
  requestedStart: [number, number];
  snappedStart: [number, number];
  targetDistanceMeters: number;
  actualDistanceMeters: number;
  durationSeconds?: number;
  ascentMeters?: number;
  descentMeters?: number;
  coordinates: Array<[number, number] | [number, number, number]>;
```

```
instructions: RouteInstruction[];
surfaceSummary?: SurfaceSummary;
wayTypeSummary?: WayTypeSummary;
provider: "openrouteservice";
profile: "foot-walking" | "foot-hiking";
metrics: RouteMetrics;
rawMetadata?: Record<string, unknown>;
}
```

Do not retain the entire raw upstream response in long-term local storage.

Validate the shape of provider responses defensively. A successful HTTP response containing invalid geometry must become a readable application error, not a crash.

11. Geometry utilities

Implement browser-side geospatial utilities as pure, tested TypeScript functions.

Include:

- Haversine distance
- Total polyline distance
- Route resampling
- Local equirectangular projection
- Coordinate quantization
- Closure distance
- Centroid calculation
- Radial-distance statistics
- Repeated-edge analysis
- Candidate similarity
- Distance error
- Route scoring
- Route bounds
- Distance from current GPS position to route

Use small, modular Turf packages only where they improve reliability. Prefer transparent custom utilities for scoring logic.

Resampling

Resample route geometry approximately every 20 metres before repeat analysis.

Preserve the first and last coordinates.

Avoid producing pathological quantities of points. Apply a reasonable hard cap and document it.

Repeat detection

The user's core requirement is to minimize running the same road more than once, particularly in the same direction.

Use this approximate, deterministic approach:

1. Project resampled coordinates into local metre-based coordinates around the route's mean latitude.
2. Quantize projected points into approximately 15–20 metre grid cells.
3. Create a directed key for each consecutive grid-cell transition.
4. Create an undirected key by sorting the two cells.
5. Ignore zero-length transitions within the same cell.
6. Measure:
7. Directed repeated distance: traversal distance beyond the first use of a directed edge
8. Undirected repeated distance: traversal distance beyond the first use of an edge in either direction
9. Divide each by total resampled route distance.

Expose:

```
interface RepeatMetrics {  
  directedRepeatMeters: number;  
  directedRepeatRatio: number;  
  undirectedRepeatMeters: number;  
  undirectedRepeatRatio: number;  
}
```

The displayed **Repeated route** percentage should use the undirected ratio because both same-direction repetition and out-and-back repetition matter to runners.

Use the directed ratio as an additional scoring penalty.

A repeated-route percentage under 10% is excellent. Up to 20% is acceptable. More than 20% should display a warning, but it must not make the application unusable in constrained geography.

Clearly label repeat detection as an estimate.

Candidate similarity

Calculate geometric similarity using the Jaccard similarity of each candidate's undirected quantized-edge set.

Treat candidates as near-duplicates when similarity is at least 0.75.

Clockwise and counterclockwise traversal of substantially the same physical route should count as duplicates.

12. Route shape and scoring

Calculate:

```
interface RouteMetrics {  
  distanceErrorPercent: number;  
  closureDistanceMeters: number;  
  directedRepeatPercent: number;  
  repeatedRoutePercent: number;  
  compactnessScore: number;  
  distanceScore: number;  
  repeatScore: number;  
  closureScore: number;  
  overallScore: number;  
  quality: "excellent" | "good" | "compromised";  
  warnings: string[];  
}
```

Use a stable, documented formula.

Suggested formula:

```
distanceScore = clamp(100 - distanceErrorPercent * 20, 0, 100);  
  
repeatScore = clamp(  
  100  
    - repeatedRoutePercent * 3  
    - directedRepeatPercent * 1.5,  
  0,  
  100  
);  
  
closureScore = clamp(100 - closureDistanceMeters / 2, 0, 100);
```

For compactness:

1. Calculate the centroid of resampled route points.
2. Calculate each point's distance from the centroid.
3. Calculate the coefficient of variation of the radial distances.
4. Convert lower radial variation into a higher 0-100 compactness score.
5. Handle degenerate routes safely.

Overall score:

```
overallScore = round(
  0.35 * distanceScore +
  0.40 * repeatScore +
  0.10 * closureScore +
  0.15 * compactnessScore
);
```

The dominant practical ranking criteria must be:

1. Repeated route
2. Distance accuracy
3. Geometric distinction from other candidates
4. Closure
5. Compactness

Quality labels:

- Excellent:
 - Distance error no more than 3%
 - Repeated route no more than 10%
- Good:
 - Distance error no more than 5%
 - Repeated route no more than 20%
- Compromised:
 - Outside either good threshold

Create warnings for:

- Distance outside tolerance
- More than 20% repeated route
- Substantial out-and-back travel
- Route not closing near the start
- Unknown elevation
- Unknown surface
- Route using steps when detected
- Trail route with incomplete surface data

Do not describe a route as safe. Routability is not a safety guarantee.

13. Elevation and surface information

Use provider summary ascent/descent where available.

If geometry contains elevations but ascent is missing, calculate ascent client-side:

- Sum positive elevation changes.
- Apply a small noise threshold to reduce GPS/elevation jitter.
- Document the method.

Show:

- Elevation gain
- Elevation loss when useful
- A compact elevation profile for the selected route, implemented with SVG or a lightweight custom component
- No heavy charting dependency

Use openrouteservice surface extras to estimate:

- Paved percentage
- Unpaved percentage
- Unknown percentage
- Dominant surface

Keep surface-ID mappings in a tested data module. Do not imply perfect accuracy.

The route cards should still work when elevation or surface extras are absent.

14. User interface

Build a polished, map-first interface optimized for one-handed phone use.

Overall layout

- Full-viewport map
- Compact top bar
- Bottom sheet for controls and route cards
- Respect iPhone and Android safe-area insets
- No horizontal page overflow
- Large touch targets
- Outdoor-readable contrast
- Responsive desktop layout without compromising mobile UX

Top bar

Include:

- LoopRoute wordmark
- Location status
- Recenter control
- Theme/settings control
- Small offline indicator when applicable

Bottom sheet states

Support:

- Locating
- Ready to configure

- Generating
- Results
- Selected route
- Live follow
- Error

The sheet should be draggable or support collapsed/expanded states without requiring an overly complex animation library.

Configuration controls

Include:

- Distance presets
- Custom distance
- Route type:
 - Road
 - Mixed
 - Trail-friendly
- Avoid steps switch
- Generate routes button
- Address/place search
- “Set start on map” guidance

Make clear that route types are preferences based on available map data.

Do not expose unsupported “avoid hills” or guaranteed-paved controls in version one. The data architecture may leave room for them later.

Route display

Display all successful candidates simultaneously:

- Selected candidate: thick, prominent line
- Other candidates: thinner and visually subdued
- Route start marker
- Requested-position marker when different from snapped start
- Dashed snap connector when applicable
- Fit map bounds with padding that accounts for the bottom sheet

Route lines must remain distinguishable in light and dark mode and for common forms of colour-vision deficiency. Do not rely only on colour; use line width, opacity, and card labels.

Route cards

Show up to three cards. Each card should include:

- Route number/name
- Actual distance
- Difference from requested distance

- Estimated duration
- Elevation gain
- Repeated route percentage
- Surface summary
- Quality badge
- Overall score
- Warning indicator where applicable

Allow the user to select a card and zoom to its route.

Use generated neutral names such as:

- Route A
- Route B
- Route C

Do not invent scenic names without data.

Empty and error states

Provide clear language for:

- Location denied
- No start selected
- No route found
- Upstream quota reached
- Routing provider unavailable
- Invalid provider response
- Offline
- API key not configured
- All candidates were near-duplicates
- Only one or two candidates could be produced

Never show raw stack traces or upstream credentials.

15. Address and map-based start selection

Implement an explicit search form:

- Text input
- Search button
- Enter submits
- No network request while typing
- Loading and empty-result states
- Up to five accessible result options
- Selecting a result moves the map and start marker

Map interactions:

- A desktop click may set the start after the user activates a “Set start” mode
- A mobile long press may set the start

- The marker is draggable
- Moving the start invalidates existing route results after a clear UI transition
- Do not silently retain routes calculated for a previous start point

16. Estimated running time

Let the user set an optional pace in settings.

Defaults:

- Metric: 6:00 min/km
- Imperial: equivalent min/mile

Store the pace locally.

Estimate running time from actual route distance and the selected pace. Do not rely on openrouteservice walking duration as a running-time estimate.

Label it **Estimated run time**.

17. Selected-route actions

Provide:

- Start following
- Export GPX
- Share route
- Generate another
- View directions
- Clear route

GPX export

Generate GPX entirely in the browser.

Requirements:

- GPX 1.1
- UTF-8
- Track name includes app name and distance
- Include latitude and longitude
- Include elevation where available
- Include metadata indicating generation by LoopRoute
- Escape XML correctly
- Use a Blob and object URL
- Use a sensible filename
- Revoke object URLs

Add unit tests for XML escaping and route output.

Share link

Because there is no backend route database, create a reproducible planning link containing:

- Approximate or exact start coordinate
- Target distance
- Route mode
- Avoid-steps setting
- Selected seed
- Display units

Default to privacy-preserving sharing:

- Round start coordinates to approximately three decimal places
- Explain that this obscures the exact start and may slightly change the regenerated route

Offer an explicit “Include precise start” option with a warning that it may reveal a home, hotel, or other private location.

On opening a shared link:

- Parse and validate all parameters
- Prioritize the shared start rather than immediately replacing it with GPS
- Show a banner indicating that a shared route plan was opened
- Offer “Use my current location”
- Regenerate using the shared seed
- Explain that the route may differ if underlying map data or routing behavior has changed

Use the Web Share API when available, with clipboard fallback.

18. Live-follow mode

Implement basic route-following, not full navigation.

When the user chooses **Start following**:

- Request continuous location updates using `watchPosition`
- Show a current-position marker
- Optionally orient the marker using available heading data
- Keep the selected route visible
- Provide a recenter/follow toggle
- Display:
 - Distance completed estimate
 - Remaining route distance estimate
 - Straight-line distance to route
 - Off-route warning when more than a sensible threshold, such as 50 metres
- Do not automatically reroute
- Do not make additional `openrouteservice` calls
- Do not provide spoken instructions
- Allow the user to stop tracking

- Stop `watchPosition` on exit and component teardown

Use the Screen Wake Lock API where supported:

- Request it only after user interaction
- Release it when live-follow mode ends or the page becomes hidden
- Fail gracefully when unsupported

Include an explicit notice:

“Map and route data may be incomplete or inaccurate. Stay aware of traffic, access restrictions, terrain and local conditions.”

19. Turn instructions

Normalize provider instructions and expose them in an accessible, collapsible list for the selected route.

Show:

- Instruction text
- Distance to next manoeuvre or segment
- Sequential numbering
- A simple icon where the instruction type can be reliably mapped

Do not attempt spoken navigation.

20. PWA and offline behavior

Make the application installable.

Include:

- Web app manifest
- App name and short name
- Theme colours
- Standalone display
- Appropriate icons
- Apple touch icon
- Service worker
- Installability metadata

Cache:

- Application shell
- Versioned static assets
- Locally bundled icons and fonts

Do not cache:

- `/api/route`

- `/api/geocode`
- Precise location-bearing API responses
- Openrouteservice responses in the service worker

Map tiles may follow normal browser/provider caching behavior, but do not build a custom bulk offline map downloader.

When offline:

- The app shell should load when previously visited
- The last locally stored selected route may be displayed
- Route generation and geocoding must be disabled
- Show a clear offline message
- Live GPS may continue displaying position over previously available content, but do not promise offline basemap availability

21. Local persistence and privacy

Use browser storage only.

Store:

- Display unit preference
- Theme preference
- Pace
- Last chosen route type
- Avoid-steps preference
- Last start point
- At most the most recently selected normalized route
- A flag recording dismissal of the safety notice

Do not store:

- Continuous GPS history
- Live-follow breadcrumb trails
- Multiple precise location histories
- Search history
- API responses containing unnecessary provider metadata

Provide a **Clear local data** action.

Use a versioned storage schema and handle corrupted or old data safely.

Do not add analytics by default.

Do not add accounts.

Do not add cookies except those intrinsically required by the hosting platform.

22. Internationalization

Provide English and Swedish interfaces.

- Detect the browser language initially
- Fall back to English
- Allow changing language in settings
- Keep translations in typed dictionaries
- Avoid embedding user-facing strings throughout components
- Format numbers and units with `Intl`
- Use:
 - English: “Repeated route”
 - Swedish: “Upprepad sträcka”
- Ensure Swedish translations are natural rather than mechanically literal

Route instructions returned by the provider may remain in the supported request language or fall back to English. Pass a supported language parameter where appropriate.

23. Theme and design

Support:

- Light mode
- Dark mode
- System mode

Visual direction:

- Minimal
- Modern
- Outdoors-oriented
- Map-first
- Serious enough to be useful but not corporate
- No excessive gradients
- No glassmorphism that harms readability
- Restrained animation
- Strong hierarchy
- High contrast

Use a locally available system-font stack rather than fetching a commercial font.

Respect `prefers-reduced-motion`.

24. Accessibility

Meet WCAG 2.1 AA principles where practical.

Include:

- Semantic HTML
- Keyboard-accessible controls
- Visible focus states
- Screen-reader labels for icon-only buttons
- Appropriate dialog and bottom-sheet semantics
- Focus management
- Escape behavior for dialogs
- Status messages using suitable live regions
- No information communicated by colour alone
- Minimum touch target sizes
- Sufficient contrast
- Reduced-motion support

The map itself is not fully accessible, so all essential route information and actions must also be available outside the map.

25. Security

Implement:

- Strict same-origin API use
- Input validation in every Function
- Allow-listed upstream hosts
- No user-controlled upstream URL
- No open proxy behavior
- No secrets in frontend environment variables
- No coordinate or address logging
- `Cache-Control: no-store` on API responses
- Safe error responses
- Request method restrictions
- Reasonable request-size restrictions
- Dependency audit review

Add security headers through `_headers` or equivalent:

- Content-Security-Policy compatible with MapLibre and OpenFreeMap
- `X-Content-Type-Options: nosniff`
- `Referrer-Policy`
- `Permissions-Policy` allowing geolocation only for self
- Frame protection using CSP `frame-ancestors`
- A reasonable `Cross-Origin-Opener-Policy` if it does not break map functionality

CSP must allow only the origins actually required for:

- Same-origin application code
- OpenFreeMap styles, tiles and sprites
- MapLibre web workers or blobs when genuinely required

The browser must never contact openrouteservice directly.

Do not log the authorization header.

26. Attribution and legal notices

Keep visible map attribution enabled.

Include appropriate attribution for:

- OpenStreetMap contributors
- OpenFreeMap
- OpenMapTiles where required
- openrouteservice

Add an About/Attribution panel explaining:

- Map and routing data may be incomplete or inaccurate
- Route suggestions do not establish legal access or physical safety
- Users must follow local laws, signs and conditions
- The app is a free hobby project
- External free services can be unavailable or rate-limited

Do not hide attribution behind an inaccessible interaction.

27. Application state

Use a clear state model rather than scattered booleans.

A reducer or small state machine should represent major stages such as:

```
type AppStage =  
  | "locating"  
  | "ready"  
  | "generating"  
  | "results"  
  | "following"  
  | "offline"  
  | "error";
```

Separate:

- Persistent preferences
- Current start point
- Request-generation state
- Candidate routes
- Selected route
- Live-follow state
- Transient UI state

Prevent race conditions:

- Assign each generation action a request ID
- Ignore late responses from superseded generations
- Abort prior requests
- Do not mix candidates from different starts or target distances

28. Suggested source structure

Use a coherent structure similar to:

```
src/  
  app/  
    App.tsx  
    appReducer.ts  
    appTypes.ts  
  components/  
    map/  
    controls/  
    routes/  
    follow/  
    settings/  
    common/  
  config/  
    app.ts  
    map.ts  
  hooks/  
    useGeolocation.ts  
    useLivePosition.ts  
    useNetworkStatus.ts  
    useWakeLock.ts  
    usePreferences.ts  
  i18n/  
    en.ts  
    sv.ts  
    index.ts  
  providers/  
    routing/  
      types.ts  
      openRouteService.ts  
    geocoding/  
      types.ts  
      openRouteService.ts  
  services/  
    routeGeneration.ts  
    share.ts  
    gpx.ts  
    storage.ts  
  geo/
```

```
distance.ts
projection.ts
resample.ts
repeats.ts
similarity.ts
scoring.ts
elevation.ts
bounds.ts
routeDistance.ts
types/
  route.ts
  api.ts
styles/
test/
functions/
  api/
    route.ts
    geocode.ts
    health.ts
  lib/
    validation.ts
    responses.ts
    ors.ts
public/
  icons/
  _headers
tests/
  e2e/
```

Adjust when needed, but maintain separation of concerns.

29. Testing requirements

Unit tests

Thoroughly test:

- Haversine distance
- Polyline distance
- Resampling
- Projection and quantization
- Directed repetition
- Undirected repetition
- A clean circle
- An out-and-back route
- A route repeating the same direction
- A figure-eight route
- Closure distance
- Candidate similarity independent of travel direction

- Distance tolerance
- Scoring and quality labels
- Elevation gain noise filtering
- Surface summary parsing
- GPX generation and escaping
- Share-link parsing and validation
- Storage schema migration
- Route point-count selection

Use realistic coordinate fixtures.

Component tests

Test:

- Distance selection
- Custom distance validation
- Location-denied fallback
- Generating state
- Successful candidate cards
- Partial candidate results
- Quota error
- Offline state
- Route selection
- Accessibility names
- Language switching
- Unit switching

Function tests

Extract validation and request-building logic into pure functions and test:

- Invalid coordinates
- Oversized distance
- Invalid seed
- Invalid mode
- Unsupported method
- Missing API key
- Correct upstream profile and options
- API key is not included in returned data
- Safe error mapping

End-to-end tests

Use Playwright with provider requests mocked at the browser/API boundary.

Cover:

1. GPS success and generation of three routes
2. GPS denial followed by address selection

3. Map-based start selection
4. Selecting a route
5. GPX export
6. Shared route parameters
7. Offline UI
8. Generate-another replacing one card
9. Mobile viewport layout
10. Live-follow mode with mocked geolocation updates

Tests must not depend on the live openrouteservice API.

30. Developer experience

Provide npm scripts including:

```
{
  "dev": "...",
  "build": "...",
  "preview": "...",
  "dev:cloudflare": "...",
  "typecheck": "...",
  "lint": "...",
  "format": "...",
  "format:check": "...",
  "test": "...",
  "test:watch": "...",
  "test:e2e": "...",
  "verify": "..."
}
```

`npm run verify` must run at least:

- Type checking
- Linting
- Unit/component tests
- Production build

Provide a reliable local Cloudflare command that runs the Pages Functions with the built frontend. If true hot-reload full-stack development is awkward, document a clear two-command or rebuild workflow instead of providing a broken script.

`.dev.vars.example`:

```
ORS_API_KEY=replace_with_your_openrouteservice_key
```

Document how to create `.dev.vars` locally and how to add the secret in Cloudflare Pages project settings.

31. GitHub Actions

Create CI triggered on pushes and pull requests.

Use:

- Current supported Node LTS
- npm cache
- `npm ci`
- `npm run verify`
- Playwright installation and E2E execution in a separate job if appropriate

Do not deploy from CI unless deployment credentials are explicitly configured.

32. Cloudflare deployment

The README must explain:

1. Create a free openrouteservice account and API key.
2. Create a Cloudflare Pages project.
3. Connect the repository or upload through Wrangler.
4. Set the build command.
5. Set the output directory to `dist`.
6. Add `ORS_API_KEY` as an encrypted production and preview secret.
7. Deploy.
8. Verify `/api/health`.
9. Test location permission over HTTPS.
10. Confirm attribution and CSP behavior.

Do not require a custom domain.

Do not perform an actual production deployment unless valid credentials and an existing authorized project are already available in the environment.

33. README requirements

Write a detailed README containing:

- Product overview
- Screens/features
- Architecture diagram using Mermaid
- Why route computation is external
- Why scoring is client-side
- Provider abstraction
- Local setup
- API-key setup
- Development commands
- Testing

- Cloudflare deployment
- Free-tier and quota limitations
- Approximate API use:
- Three calls per initial generation
- Up to two bounded replacements
- One call per “Generate another”
- Privacy model
- Local storage behavior
- Security model
- Attribution
- Known limitations
- Why “road,” “mixed,” and “trail” are preferences rather than guarantees
- Why repeated-route detection is approximate
- Why shared routes may change later
- How to replace the routing provider
- Troubleshooting:
- Missing key
- 401
- 429
- Location denied
- CSP failures
- Blank map
- No routes found
- Cloudflare Function errors

State plainly that free third-party APIs may change their quotas, terms, or availability.

34. Quality requirements

The finished application must:

- Build without TypeScript errors
- Pass linting
- Pass unit and component tests
- Pass the implemented E2E suite
- Contain no actual secrets
- Contain no major TODO placeholders
- Be usable on a 375 px-wide mobile viewport
- Be installable as a PWA
- Function without accounts
- Generate routes only through the same-origin proxy
- Preserve map attribution
- Remain usable when geolocation is denied
- Handle fewer than three route results
- Handle quota exhaustion cleanly
- Avoid server-side location storage
- Avoid unnecessary API calls
- Stop live GPS tracking correctly
- Export a valid GPX file
- Include English and Swedish interfaces

- Include light, dark and system themes
- Include accessible controls and readable non-map route information

35. Execution procedure

Follow this procedure autonomously:

1. Inspect the current directory and available tools.
2. Create a concise internal implementation plan.
3. Scaffold the application.
4. Install dependencies.
5. Build the domain models and pure geospatial utilities first.
6. Add unit tests for those utilities.
7. Implement the Cloudflare Functions and pure validation tests.
8. Implement provider adapters.
9. Implement the map and main workflow.
10. Implement route cards, scoring, GPX, sharing and persistence.
11. Implement live-follow mode.
12. Implement PWA support, localization, accessibility and themes.
13. Add component and E2E tests.
14. Add security headers and deployment configuration.
15. Write the README and AGENTS.md.
16. Run formatting.
17. Run `npm run verify`.
18. Run Playwright tests.
19. Run a production-build preview or equivalent smoke test.
20. Inspect the final diff and repository for:
 - Exposed secrets
 - Broken imports
 - Missing error handling
 - Unused dependencies
 - Accessibility regressions
 - Excessive bundle size
 - Accidental direct calls to openrouteservice
21. Fix all issues found.
22. Provide a final report listing:
 - What was built
 - Important architectural decisions
 - Verification commands run and their results
 - Any remaining limitations
 - Exact local-run and Cloudflare-deployment steps

Do not merely describe what the code should do. Implement it.